Cliff Lin

SOLUTION ARCHITECT

Internal Systems · Software Engineering · Cloud & Platform

cliffium.github.io

從 Infrastructure、Software Engineering 到 Systems Analysis，累積超過 10 年企業 IT 與系統開發經驗。 專注於將實際業務需求轉換為可落地、可維護、可持續演進的技術方案。

個人簡介 / Profile

擅長從使用者與業務需求出發，釐清真正需要解決的問題，再將需求轉換為 workflow、data model、API、 UI/UX、application architecture 與 deployment architecture。除了系統分析與架構設計，也具備實際的 軟體開發、系統整合、Cloud 與 Kubernetes 平台建置能力。

近年亦將 AI 納入實際工程流程：先建立 architecture、coding conventions、shared modules 與 implementation constraints，再利用 AI 協助實作並進行 code structure 與 architecture review。

技術能力 / Technical Skills

Backend & Application

Golang (Gin · GORM) | C# / .NET (MVC · WinForms · SignalR) | SQL / RDBMS (Schema · Query · Stored Procedure) | React / Next.js (AI-assisted production delivery)

Integration & Security

REST API / Integration (System boundaries · data flow) | SAP Business One (Workflow · inventory · integration) | OAuth2 / JWT / SSO (Azure AD · token flow)

Cloud & Platform

Kubernetes (3 control planes · 3 workers) | Helm (Reusable deployment patterns) | AWS (EC2 · S3 · MediaConvert · Amplify) | Linux (Server · runtime · Bash) | Hyper-V (Virtualization platform) | Networking & Infrastructure (HAProxy · Ingress · DNS)

Engineering

Git (Version control · team workflow) | AI-assisted Engineering (Architecture-guided implementation)

Supporting Technologies

Backend & Data: Gin, GORM, Casbin, MS SQL, MariaDB / MySQL, SAP HANA

Frontend & .NET: JavaScript / jQuery, CSS / SCSS, Bootstrap, ASP.NET MVC, WinForms, SignalR

Infrastructure & Operations: Ingress, HAProxy, NFS, Veeam, Windows Server, VMware, Bash

Engineering & Enterprise: Jenkins, SonarQube, Swagger / OpenAPI, Joget, Java, Qt, Matlab

語言與學歷 / Languages & Education

Chinese: 母語

English: 可閱讀技術文件，並進行工作與技術溝通 · 歷史 TOEIC 720

Japanese: 基礎日常對話與閱讀，可理解部分日文操作文件

2008 - 2013 | 元智大學 Yuan Ze University | Bachelor of Science in Computer Science / 資訊工程學士

工作經歷 / Professional Experience

全智通機器人股份有限公司 | 2020.03 - Present

System Analyst

Role evolution since 2024.10 · Solution Architecture

- 從需求到 deployment，自主規劃內部 solution 的 workflow、data、API、UI/UX 與技術架構。
- 建立內部流程系統與共用 Golang services，涵蓋 SAP、權限、報表、排程與系統整合。
- 從零建置 6-node Kubernetes application platform，統一內部應用程式的部署環境。
- 導入 architecture-guided AI-assisted engineering workflow，並部署 DeepWiki 進行企業 AI 應用驗證。

2024.10 - Present · Solution Architecture & Internal Systems Development

- 內部流程系統目前服務約 20 名使用者，涵蓋約 4-5 個部門，包含權限管理、SAP 表單、庫存視覺化與簽核流程。
- 以 Golang 建立 Casbin 權限、JWT/API 驗證、SAP / Shopfloor integration、ETL、排程報表與通知服務。
- 將部分多步 SAP 操作改為一次送單後由後端完成；人工日報改為夜間自動產生並寄送。
- 將 DeepWiki 部署至 Kubernetes，透過 OpenRouter 比較 LLM 輸出並調整 prompt。
- 設計 AWS 多媒體展示平台，並將官網由 EC2 遷移至 Amplify，降低約 60% 雲端費用。

Kubernetes Internal Application Platform

在既有 Hyper-V 環境上建立一致的 application runtime，使用 Helm 管理 deployment pattern，並設計 probes 與 node replacement procedure。

HAProxy > Ingress > 3 Control Planes + 3 Worker Nodes > Hyper-V

2020.03 - 2024.09 · Systems Analysis & Manufacturing Systems

- 與 Software RD、Hardware RD、產線與使用部門訪談，將實際作業需求轉換為 workflow、data specification 與 application design。
- 持續改善 Shopfloor / MES，包含生產流程、工時統計、測試資料檢查、工單狀態與報表。
- 定義統一資料上傳 API 與文件，並協助海外工程師導入。
- 執行 Order System PoC，負責訂單、產能、出貨與 SKU 整合流程設計。

自由職業者 | 2019.12 - 2020.02

Freelance Software Engineer

直接與製造業客戶訪談，從現場 workflow、UI 設計一路完成應用程式開發，建立 end-to-end delivery 經驗。

綠界科技股份有限公司 | 2019.06 - 2019.11

Software Engineer

維護財政部電子發票 integration API、產品 SDK 與會員後台，並撰寫新專案程式架構文件。

網訊電通股份有限公司 | 2015.04 - 2019.06

MIS Engineer → Software Engineer

Career transition · Infrastructure → Software

- 由企業 MIS 工作中的自動化與工具開發，自然轉任專職 Software Engineer。
- 使用 C# / .NET MVC、REST API、SQL 與 JavaScript / jQuery 開發內部及客戶系統。
- 早期負責超過 2,000 台 PC、30+ servers 與 4 個跨國分公司的 IT 環境。

2017.02 - 2019.06 · Software Engineer

- 從零規劃員工權限紀錄系統，並與既有權限申請流程整合。
- 參與外部活動網站與媽媽教室平台，負責前後端流程、付款狀態控制與每日訂單對帳。
- 以非同步讀取與單頁式操作改善既有客服系統效能與 UX。

2015.04 - 2017.02 · MIS Engineer

- 維運 2,000+ 使用者 PC、30+ servers 與 4 個跨國分公司的企業網路。
- 以 Batch、Bash 與自行開發的 C# WinForms 工具改善維運，並以 SignalR 建立端點與管理端雙向溝通。

精選專案 / Selected Projects

Internal Workflow System

PROFESSIONAL PROJECT | React / Golang / SAP B1 / Kubernetes

以 React / Next.js 建立的內部 Web System，整合使用者權限、SAP 表單申請、庫存查詢與視覺化，以及簽核流程。

Role & Ownership: 負責需求分析與 solution design，自主規劃 workflow、DB schema、API、UI/UX、technology stack 與 deployment architecture。

Representative Impact: 將部分多步驟 SAP 操作簡化為一次送單後由後端自動完成；每日人工報表改為夜間自動產生並寄送。

UX Decision: 將原本以數量與表格為主的庫存資訊重新設計為包含 heatmap 的視覺化介面，降低資訊判讀成本。

Demo: https://dancing-basbousa-e6f2a8.netlify.app

Manufacturing MES / Shopfloor System

PROFESSIONAL PROJECT | REST API / SQL / C# / .NET / Joget

將製造現場的生產流程、測試資料、生產履歷與工單資訊系統化整合，並定義跨工具共用的資料介面。

Collaboration: 與 Software RD、Hardware RD 與產線人員溝通，分析現場 workflow，定義資料交換方式與系統流程。

Integration: 定義統一 REST API，使不同部門、不同程式語言開發的測試工具能以一致方式上傳測試資料。

Operations: 建立工時統計、測試資料檢查、工單狀態、報表、生產履歷與物料使用資訊。

AWS Multimedia Streaming Platform

PROFESSIONAL PROJECT | MediaConvert / S3 / Amplify / CloudFront / Route53

依業務展示需求建立可持續維護的多媒體平台，將內容更新流程與 application code 解耦。

Architecture: Source Video → MediaConvert → Streaming Format → S3 → CloudFront / Amplify → Corporate Domain。

Key Decision: 影片排列與內容由 S3 上的 JSON 定義，新增影片時依 SOP 更新媒體與 metadata，不需要修改 application。

Cloud Optimization: 另將公司官方網站由 EC2 遷移至 Amplify，在符合既有需求的前提下降低約 60% 雲端費用。

Demo: https://knowledge.aeolusbot.com/

Travel Planning Web Application

PERSONAL PROJECT | Information Architecture / UX / AI Collaboration

把真實旅行中的模糊需求，透過 Systems Analysis、Information Architecture、UI/UX 與 AI-assisted Engineering 轉換為實際可使用的行程 Web App。

Purpose: 驗證能否把生活中的真實問題，透過 SA 思維整理成系統，而不是單純製作技術 Demo。

Design Focus: 以旅途中快速掃讀為核心，整理每日行程、交通、餐廳、景點、注意事項與低頻資訊。

Process: Requirement → Information Architecture → UX / Workflow → AI-assisted Implementation → Actual Usage。

Demo: https://sprightly-paletas-b2c7c7.netlify.app/

VTuber Automatic Subtitle & Translation System

PERSONAL R&D · IN PROGRESS | Python / Whisper / Local LLM / CUDA

研究將長時間日文 VTuber 影片自動轉換為日文 / 繁中雙語字幕的 AI pipeline，重點放在可重現評估與架構取捨。

Pipeline: YouTube Audio → Japanese ASR → Context Enrichment → Correction Research → Translation → Bilingual Subtitle。

Evaluation: 使用 CER / WER、固定測試資料與第二支影片進行 cross-video validation，避免只對單一素材調參。

Engineering Principle: 若額外 AI correction stage 無法證明穩定淨改善，且可能破壞原本正確字幕，優先移除而非為了架構完整度保留。